ISTA 322 Homework 1

Welcome to your first homework! This one is focused on just practicing some of the exercises covered in the last coding lesson. There are also some more open-ended questions with elements that I didn't demonstrate in that lesson... they're structurally similar, but you might need to google a thing or two to figure out the correction function.

You need to add your own code blocks to answer any of the coding questions. Also, at the end of some sections I have a 'questions' section. Add a text cell right below and enter your answers.

Submission Instructions

1) First create a copy of this notebook in your drive and rename it so that it contains the course number (ISTA322), the semester (Sp22 for Spring 2022, Su22 for Summer 2022, and Fa22 for Fall 2022), the assigment code (HW1 for this assignment) and your name.

```
e.g. my copy for Spring 2023 would be ISTA_322_Sp23_HW1_dan_charbonneau)
```

2) When you are ready to submit. Prepare three files: the python file (File->Download->Download .py), the notebook file (File->Download->Download .ipynb), and **PDF** version of your notebook (**after running all cells**). You should be able to create a pdf through File -> print -> Destination: save as pdf

3) Create a new folder named firstname_lastname_hw1 (e.g my folder would be dan_charbonneau_hw1) put all three files in it. Compress (Zip) the folder you created with the files inside of it and submit this .zip file to D2L

incorrect filenames or submission formats will result in a loss of 50% of your grade

Loading and Importing

First thing you need to do is load up your packages and then bring in the data.

This dataset contains daily values for Amazon's stock. This includes opening, closing, high price, low price, and also the amount of stock traded.

```
import pandas as pd
```

```
import pandas as pd
import datetime
# also import matplotlib.pyplot and numpy with the proper aliases
import numpy as np
import matplotlib.pyplot as plt
```

```
# Bring in your data. You just need to run this cell.
price = pd.read_csv("https://docs.google.com/spreadsheets/d/1lCkFZhz-NGTuE1ZilzJA_ZYBZsbC
price.head()
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	1997-05-15	2.437500	2.500000	1.927083	1.958333	1.958333	72156000
1	1997-05-16	1.968750	1.979167	1.708333	1.729167	1.729167	14700000
2	1997-05-19	1.760417	1.770833	1.625000	1.708333	1.708333	6106800
3	1997-05-20	1.729167	1.750000	1.635417	1.635417	1.635417	5467200
4	1997-05-21	1.635417	1.645833	1.375000	1.427083	1.427083	18853200

Exploring the whole dataset

Now make some code cells to explore the whole dataset. I want you to do the following:

- Get the number of rows and columns
- Get the datatypes of each column
- Look at the first five rows
- Look at the last five rows
- Look at summary statistics

Questions

Write down Pandas instruction to answer these queries.

- How many rows are in this dataset?
- Do any datatypes need to be converted?
- What was the mean and all time high opening stock price?

Q1: [2 points] How many rows are in this dataset?

```
#do not change the function name or number of parameters
## Q1 Your function starts here
```

```
## Q1 Your function starts here
def price_number_of_row():
  return 0 ##replace this line with return of the correct statement
## Q1 Your function ends here - Any code outside of these start/end markers won't be grad
```

As an example, I will provide the answer to this question:

```
def price_number_of_row_key():
  return price.shape[0];
```

```
print(price_number_of_row_key())
```

```
5852
```

Question 2: [2 points] Convert the datatype of 'Date' to an appropriate type

```
## Q2 Your function starts here
def convert_date_type():
  price["Date"] = pd.to_datetime(price["Date"])
## Q2 Your function ends here - Any code outside of these start/end markers won't be grad
```

I give you the grader statement for this one too:

```
convert_date_type();
print(price.Date.dtype);
```

```
datetime64[ns]
```

Question 3: [2 points] What is the mean of all opening stock price?

```
## Q3 Your function starts here
def mean_of_opening():
  return price["Open"].mean(); #replace this with the correct statement
## Q3 Your function ends here - Any code outside of these start/end markers won't be grad
```

```
print(mean_of_opening())
```

```
377.4695570299043
```

Question 4: [4 points] What is the daily min volume for trades after 2010 (including 2010-1-1)

```
## Q4 Your function starts here
def min_daily_volume_after_2010():
  #hint first construct a new dataframe by limiting the price for dates greater than 2010
  #print(price.Date.dtype) <-- Debugging
  dates_gt_2010 = price["Date"] >= np.datetime64('2010')
  min_price_past_2010 = price[dates_gt_2010].min()

  return min_price_past_2010["Volume"] #replace this with your answer
## Q4 Your function ends here - Any code outside of these start/end markers won't be grad


print(min_daily_volume_after_2010())
```

```
    881300
```

Question 5: [4 points] Make a new column called up_binom. The value of up_binom is 1 if the closing price of the stock is higher (>=) than opening price, and 0 otherwise. Then find the number days the stock closed higher than openning.

```
## Q5 Your function starts here
def number_of_green_days():
  close_gt_open = price["Open"] < price["Close"]
  price["up_binom"] = close_gt_open.astype(int)
  green_days = price["up_binom"] == 1

  return green_days.sum();
## Q5 Your function ends here - Any code outside of these start/end markers won't be grad


print(number_of_green_days());
```

```
    2930
```

Question 6 [4 points] Write a function that plots the Closing price of the stock for the year of 2020. The x-axis is day and the y-axis is the Closing price. Look at the examples here for more help https://matplotlib.org/stable/gallery/index.html)

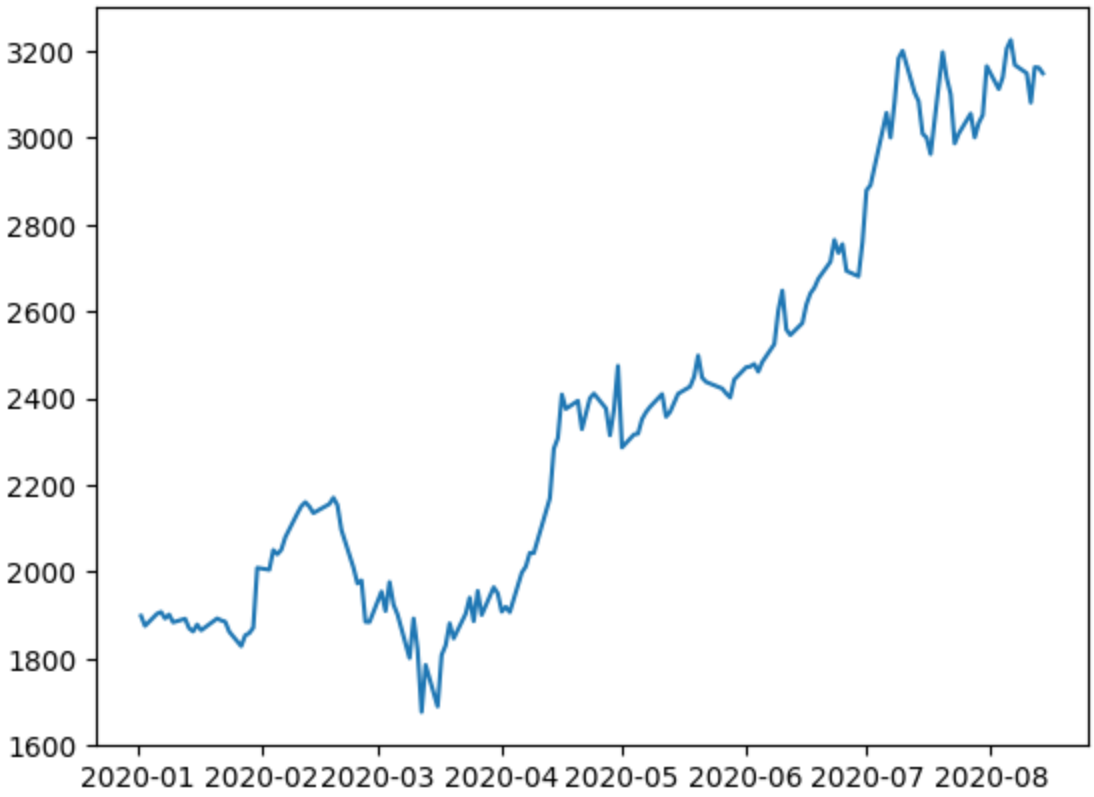
```
#plot
## Q6 Your function starts here
def plot_closing_price_2020():
  #Add your code for your plot
  time_gt_2020 = price["Date"] > np.datetime64('2020')
  time_lt_2021 = price["Date"] < np.datetime64('2021')

  prices_in_2020 = price[time_gt_2020 & time_lt_2021]

  fig, ax = plt.subplots()
```

```
  ax.plot(prices_in_2020["Date"], prices_in_2020["Close"])
  plt.show()


# function call to plot your graph
## DO NOT REMOVE THIS FUNCTION CALL OR MOVE IT OUTSIDE OF THE MARKERS
plot_closing_price_2020()
## Q6 Your function ends here - Any code outside of these start/end markers won't be grad
```



Question 7: [4 points] Word problem time!

When I was 18 I had saved up $2000 washing dishes at a local restaurant to buy a new gaming rig (a cutting edge Pentium 5 :D). This was actually right around the start of this dataset. Let's say instead of buying that computer, I had put all that money into Amazon stock, what would my total profit have been if I'd sold all of my shares on the last day of the dataset?

Let's say that I bought the shares at opening on Aug 1, 1997 and sold them at close on the last day of the dataset (don't assume that the dataset is sorted by date).

Keep in mind that these are real world data and, as such, have constraints that you need to consider: e.g., you can only buy full shares and not fractions of shares. Also keep in mind that the question is about total **profit**, which should account for the money I spent on the shares. To keep things simple, your answer can (and should for ease of grading) include fractions of cents even though that wouldn't really be possible.

Write a function ***market_value(buy_date, sell_date, starting_cash)*** that calculates the total profit from a number of shares bought at open of one date and sold at close of another date and use it to solve the word problem

```
# I'm going to make your life easier and set the date column you created as the index.
# This will make searching and extracting the values much easier
price = price.set_index('Date')


## Q7 Your function starts here
def market_value(buy_date, sell_date, starting_cash):
  buy_price = price.at[pd.to_datetime(buy_date), "Open"]
  sell_price = price.at[pd.to_datetime(sell_date), "Close"]

  num_bought = starting_cash / buy_price
  num_sold = num_bought * sell_price

  return num_sold - starting_cash; #replace this with your answer
## Q7 Your function ends here - Any code outside of these start/end markers won't be grad


last_date_in_dataset = price.index[-1] # replace with code that finds the last date in th
print(market_value('1997-08-01', last_date_in_dataset, 2000))
```

```
     2684310.4170666668
```

JSON

The last part of the assignment will have you working with some basic JSON data. The URL links to a JSON file with stats on every episode of the TV show Silicon Valley

```
# First just run this to import the data
import requests
url = 'http://api.tvmaze.com/singlesearch/shows?q=Silicon Valley&embed=episodes'
sv_json_obj = requests.get(url)
sv_json = sv_json_obj.json()
```

Viewing your JSON

Now just to look at what's in the JSON a bit

- Make a code cell that just calls the JSON we named above.
- Also run the .keys() function on the object.

```
sv_json
```

```
sv_json
sv_json.keys()
```

```
dict_keys(['id', 'url', 'name', 'type', 'language', 'genres', 'status', 'runtime',
'averageRuntime', 'premiered', 'ended', 'officialSite', 'schedule', 'rating',
'weight', 'network', 'webChannel', 'dvdCountry', 'externals', 'image', 'summary',
'updated', '_links', '_embedded'])
```

```
# Debugging
sv_json['_embedded']['episodes'][0].keys()
```

```
dict_keys(['id', 'url', 'name', 'season', 'number', 'type', 'airdate', 'airtime',
'airstamp', 'runtime', 'rating', 'image', 'summary', '_links'])
```

Questions

Based on these responses, what keys are present in the JSON. More importantly, are there any keys that don't get returned by .keys()?

```
# First, you can see the structure after moving down a level into '_embedded'
sv_json['_embedded']
```

Show hidden output

RESPONSE:

These keys are present in the JSON:
id,url,name,season,number,type,airdate,airtime,airstamp,runtime,rating,image,summary,_links.

The .keys() method doesn't return these keys: season, number, airdate, airtime, and airstamp.

Question 8: [4 points] Find the day that show premiered (ie the date that the very first episode first aired).

```
## Q8 Your function starts here
def get_show_premiered():
  pilot_episode = ''
  for episode in sv_json['_embedded']['episodes']:
    if (episode['season'] == 1) and (episode['number']) == 1:
      pilot_episode = episode
      break

  return pilot_episode['airdate']; #replace this with the correct statements
## Q8 Your function ends here - Any code outside of these start/end markers won't be grad
```

```
print(get_show_premiered());
```

```
     2014-04-06
```

Question 9: [4 points] Get the summary of a specific episode.

```
## Q9 Your function starts here
def get_summary(season, episode):
  summary = ''

  for curr_episode in sv_json['_embedded']['episodes']:
    if(curr_episode['season'] == season) and (curr_episode['number'] == episode):
      summary = curr_episode['summary']
      break

  return summary; # replace this with the correct statement
## Q9 Your function ends here - Any code outside of these start/end markers won't be grad
```

```
print(get_summary(2,5))
```

```
     <p>Gavin creates interference that hinders Pied Piper's expansion. Meanwhile, the guy
```